

Tirsdag d. 26. august

Opgave 1

Du er givet et interface `Measurable` som ser ud som følger:

```
public interface Measurable {  
    public double getMeasure();  
}
```

Lav nu en ny klasse `Chili`, som *implementerer* `Measurable`-interfacet. En `Chili`-klasse skal have felter til navn og til styrke (husk `get/set`-metoder og constructor). Metoden `getMeasure()` skal returnere chiliens styrke målt i [Scoville](#) (et tal imellem 1 og 2.000.000).

Opret en klasse `App` med en `main()`-metode.

Klassen `App` skal udover `main()`-metoden indeholde følgende statiske metoder:

```
public static Measurable max(Measurable[] objects) { . . . }  
public static double avg(Measurable[] objects) { . . . }
```

Metoden *max* skal finde det største `Object` i parameterlisten, og metoden *avg* skal finde den gennemsnitlige værdi for alle `Objects`.

I `main()` metoden tilføjes kode, der afprøver dine to statiske metoder med et array af `Chili`-objekter, og udskriv den gennemsnitlige styrke og den højeste styrke.

Opgave 2

Lav en `Customer`-klasse med fornavn, efternavn og alder. Klassen skal have en constructor samt `set-` og `get-`metoder.

`Customer`-klassen skal implementere interfacet `Comparable`. Programmér metoden `compareTo`, så en kunde kommer før en anden kunde, hvis kundens efternavn kommer før en anden kundes efternavn i henhold til den naturlige ordning på `Strings`. Hvis to kunder har samme efternavne, sammenlignes yderligere på fornavn, og hvis begge fornavne er ens, sammenlignes på alder (yngst først).

Lav en `App` klasse med en `main()` metode. Tilføj til `App` klassen en metode, der givet et array af `Customers`, returnerer den `Customer` der kommer til sidst iht. deres naturlige ordning. Metoden skal have følgende signatur:

```
public static Customer lastCustomer(Customer[] customers)
```

Tilføj endnu en metode, der givet et array af `Customers` og et `Customer`-objekt, returnerer et nyt array af kunder, bestående af de kunder, der kommer efter den angivne `Customer` i input-arrayet. Metoden skal have følgende signatur:

```
public static Customer[] afterCustomer(Customer[] customers,  
                                       Customer customer)
```

Afprøv de statiske metoder på passende vis fra `main`-metoden.

Opgave 3

Du skal nu vende tilbage til ordresystemet der blev kodet i opgave 3 den 23. august. Du skal lave de ændringer der skal til, så Discount-typen nu er implementeret som et Java interface. Se diagrammet herunder. Test at det hele stadig virker. Tilføj også en ekstra Discount-implementation der hedder AgeDiscount og test denne. AgeDiscount skal have kundens fødselsår med som paramenter ved oprettelse. Rabatten er da en procentsats svarende til kundens alder i procent.

